



Marwadi
University
Marwadi Chandarana Group



Experiment – 2 Aim :

Implementation and Time analysis of linear and binary search algorithm.

Hardware Required:

Mouse, keyboard **Software Required:**

code Blocks

a) Linear search Algorithm:



Linear Search:

- Step 1: start
- Step 2: create an array arr[100] and variable size and key
- Step 3: Enter size of the array
- Step 4: Enter array elements & print
- Step 5: sort the array using bubble sort & print sorted array
- Step 6: Enter the element to be searched & store in key.
- Step 7: set low = 0
High = size - 1
And calculate mid = (low + high) / 2
- Step 8: compare key with arr[mid]
 - If key == arr[mid] → element found
 - If key < arr[mid] → set high = mid - 1
 - If key > arr[mid] → set low = mid + 1
 - Repeat until low <= high
- Step 9: If low > high, print element not found

Program:

```
#include <iostream>
using namespace std;
```

```
int main() {
```



```
int n, target;

cout << "Enter the number of elements: ";
cin >> n;

int arr[n];

cout << "Enter " << n << " elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Enter the element to search: ";
cin >> target;

// Linear Search
for (int i = 0; i < n; i++) {
    if (arr[i] == target) {
        cout << "Element found at index " << i << endl;
        return 0;
    }
}

cout << "Element not found" << endl;

return 0;
}
```

Output:

Output

```
Enter the number of elements: 4
Enter 4 elements: 10 20 43 21
Enter the element to search: 20
Element found at index 1
```

=== Code Execution Successful ===

Time complexity:

Time complexity:

```
int i;
for(i=0; i<size; i++){
    if(arr[i]==key){
        cout<<"element found at index i"<<endl;
        break;
    }
}
if(i==size){
    cout<<"element not found"<<endl;
}
```

$\Rightarrow 1+1+n+1+n+1$
 $\Rightarrow 3n+4$
 $= O(n)$

Best case: If element found at first position
 $T(n) = O(1)$

Average case: If element found at middle
 $T(n) = O(n)$

Worst case: If element found at last position
 $T(n) = O(n)$

b) Binary Search Algorithm:



step 1: start
step 2: create an array arr[100] and variable size and key
step 3: Enter size of the array
step 4: Enter array elements & print
step 5: sort the array using bubble sort & print sorted array
step 6: Enter the element to be searched & store in key.
step 7: set low = 0
 High = size - 1
 And calculate $mid = (low + high) / 2$
step 8: compare key with arr[mid]
 → If $key == arr[mid]$ → element found
 → If $key < arr[mid]$ → set $high = mid - 1$
 → If $key > arr[mid]$ → set $low = mid + 1$
 → Repeat until $low \leq high$
step 9: If $low > high$, print element not found

Program:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n, target;
```

```
    cout << "Enter number of elements: ";
    cin >> n;
```

Name: M. Tarmisha
Enrollment number :
92460118029



```
int arr[100];

cout << "Enter elements in sorted order: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

cout << "Enter element to search: ";
cin >> target;

int low = 0;
int high = n - 1;
bool found = false;

while (low <= high) {
    int mid = low + (high - low) / 2;

    if (arr[mid] == target) {
        cout << "Element found at index " << mid << endl;
        found = true;
        break;
    }
    else if (arr[mid] < target) {
        low = mid + 1;
    }
    else {
        high = mid - 1;
    }
}

if (!found) {
    cout << "Element not found" << endl;
}

return 0;
}
```



Output:

```
Enter number of elements: 6
Enter elements in sorted order: 5
1
3
6
8
9
Enter element to search: 3
Element found at index 2
```

```
=== Code Execution Successful ===
```

Time complexity:



Time complexity:

int low, high, mid; — 1

low = 0; — 1

high = size - 1; — 1

while (low <= high) { — $\log n + 1$

mid = (low + high) / 2; — $\log n$

if (key == arr[mid]) { — $\log n$

cout << "element found" } — 1

return; — 1

}

else if (key < arr[mid]) — $\log n$

{

high = mid - 1; — $\log n$

}

else

{

low = mid + 1 — $\log n$

}

} $\Rightarrow 1 + 1 + 1 + \log n + 1 + \log n + \log n + 1 + 1 + \log n + \log n$

$= 8 \log n + 8$

$= \log n \Rightarrow O(\log n)$

Best case: $O(1)$

Average case: $O(\log n)$

Worst case: $O(\log n)$

Result:

As per the experiment to Implementation and Time analysis of linear and binary search algorithm.

Output was verified successfully according to the given experiment.

Name:M.Tarmisha
Enrollment number :
92460118029
